

Graph Buddy — an interactive code dependency browsing and visualization tool

^{1st} Krzysztof Borowski
AGH University and VirtusLab
R&D Department
Krakow, Poland
kborowski@virtuslab.com

^{2nd} Bartosz Balis
AGH University of Science and Technology
Institute of Computer Science
Krakow, Poland
balis@agh.edu.pl

^{3rd} Tomasz Orzechowski
Virtus Lab Spolka z o.o.
R&D Department
Krakow, Poland
torzechowski@virtuslab.com

Abstract—Source code comprehension of massively growing code bases is the first crucial step in today’s software development. One of the significant obstacles in reading the source code is understanding the code dependencies. To aid this process, we developed *Graph Buddy*, a tool integrated as a plugin for popular IDEs, which enables interactive visual code browsing as a graph of semantic code dependencies. To preserve the direct relation to the source code, visualize, and process only the right amount of details, we have developed *Semantic Code Graph* (SCG), an information model underlying *Graph Buddy*, capable of representing detailed code semantics. We implemented the SCG model extraction for the Scala and Java languages. To evaluate *Graph Buddy*, we conducted a user survey on a group of 10 programmers who used the tool in three programming-related tasks. The survey indicates that the tasks would be difficult to solve without the visual browsing tool. We believe that the proposed solution has a strong potential to enhance the way programmers comprehend code dependencies.

Index Terms—software visualization, semantic code graph, static analysis, scala, java

I. INTRODUCTION

Developers often need to browse through massive source code bases whose code quality tends to degrade over time [1]. Consequently, understanding and maintaining even parts of a large system becomes extremely difficult and time-consuming. There are several places where developers look for information about the system, including documentation, an issue tracking system, or a version control system. However, the ultimate knowledge base is the source code itself. Consequently, we argue that helping the programmer with code understanding increasingly becomes one of the most essential factors in software development.

Analyzing dependencies between modules, classes, methods, and other code definitions is particularly challenging, especially in older projects, where the quality of code architecture often degrades. Various visualizations and visual code browsing improvements can speed up code reading and understanding, and facilitate code maintenance. However, despite the broad academic research in this area, the adoption of code visualization solutions remains low. Designed tools tend to be short-lived prototypes, hard to set up and run, and not integrated within the developers’ IDE. Tools also suffer from usability and performance problems [2]. As such, designed solutions cannot be empirically evaluated in practice in a wide

range of projects with comprehensive user feedback. Usability and performance issues are caused not only by implementation shortcomings, but also by inadequate source code abstract representation for a given problem. This often results in low tool responsiveness and cluttered visualizations when used on real projects.

To address the above issues, we developed the *Graph Buddy* tool, a publicly available extension for popular IDEs that allows interactive visual source code browsing and dependency discovery (Section III) built on a code information model, the *Semantic Code Graph* (SCG) (Section II). We evaluate the solution in Section IV. The related work is analyzed in Section V. Section VI concludes the paper.

II. GRAPH BUDDY INFORMATION MODEL

Our goal is to provide the developer with an efficient visualization to navigate semantic code dependencies between different code units and, in the process, to reduce task cognitive overload and speed up the learning process. For object-oriented programming, that visualization would include supporting call graphs discovery, class inheritance, method overloading, or presenting class overview. To make this possible, we need the right level of information extracted from the source code. Representations such as control flow graphs or data dependency graphs [3], although suitable for various static code analysis and optimizations, are too granular (statements-level granularity) to visually represent semantic dependencies. Such visualizations look cluttered, too detailed, and hence unreadable to the end user. Moreover, processing unnecessary details results in additional performance and scalability overhead. On the other hand, representations like UML class diagrams are too strict and often lack important properties for code dependency visualization and browsing (such as call graph details). Consequently, we propose a semantics-rich abstract code information model – the *Semantic Code Graph* (SCG).

A. Semantic Code Graph definition

The *Semantic Code Graph* (SCG) is a code structure information model that is (a) semantics-rich: it represents a variety of relationships between code definitions; (b) closely

tied to the source code; (c) adaptable to various programming languages.

SCG is defined as a pair $G_{SCG} = (V_D, E_R)$ comprising:

- V_D , a set of vertices (nodes) representing all code declarations, e.g., classes, methods, variables;
- $E_R \subseteq \{(x, y) \mid (x, y) \in V_D^2\}$, a set of edges which are ordered pairs of nodes representing *relations* between nodes, e.g., call, declaration, override, inheritance, or return type.

SCG nodes and edges also have to contain the following properties: *location* – location of the related source code; node/edge *type* and *displayName* meaningful for the user. In addition, nodes and edges may contain optional properties depending on their type, e.g., information about the scope of a method or the package containing it. Such a representation allows modeling information specific for a given programming language with a common representation format constrained only to properties crucial for visualization. For the Scala language, the *types* of the nodes are: OBJECT, CLASS, TRAIT, METHOD, PARAMETER, TYPE, VALUE, VARIABLE, and *types* of the edges are defined with: CALL, DECLARATION, EXTEND, OVERRIDE, RETURN, PARAMETER and TYPE_PARAMETER. For Java it is similar, with minor adjustments made, such as the node *type* TRAIT being replaced by the INTERFACE. Similarly, the SCG definition for other languages can be adjusted to accommodate the specifics of the given language.

B. Semantic Code Graph extraction

To collect the rich code structure data modeled in the SCG, we analyze the source code to extract structural details from the parsed AST and information about the code semantics after the code symbol resolution phase. In the case of the Scala language, we created and published a compiler extension for Semantic Code Graph extraction.¹ During compilation, for each source file, the plugin generates a corresponding `[file-Name].semanticgraph` file which contains the SCG data in the Protocol Buffers format. For the Java language, we utilized the JavaParser² library which, thanks to the explicitness of the Java language, allows us to resolve symbol dependencies without actual code compilation. We work directly with the source code to generate its most precise and direct representation for further interactive visualizations. Extracting SCG for each particular language has to be implemented separately. We open sourced the SCG format³ and plan to create extensive documentation on how to contribute to the project by adding support for additional languages.

III. GRAPH BUDDY IDE'S PLUGINS

Graph Buddy aims to speed up the process of analyzing, browsing, and understanding source code dependencies through interactive visualization of the *Semantic Code Graph*.

¹<https://search.maven.org/search?q=org.virtuslab.semanticgraphs>

²<https://github.com/javaparser/javaparser>

³https://github.com/VirtusLab/graphbuddy/blob/master/proto/graph_node.proto

Graph Buddy is a plugin for Visual Studio Code⁴ and IntelliJ⁵ IDEs, publicly available and easily installable through the respective marketplaces. It works out of the box based on generated metadata (see II-B) – it is enough to open the Graph Buddy board, and the indexing process will start automatically followed by the loading and introduction screen.

A. Graph Buddy architecture

The complete Graph Buddy visualization process consists of two phases:

- 1) Generating graph data in the *.semanticgraph* format during source code static analysis (for Scala with the help of a custom compiler plugin).
- 2) Indexing and exposing graph data using the Graph Buddy IDE plugin for IntelliJ and Visual Studio Code.

Graph Buddy architecture consists of the following modules (Fig. 1):

- Separate Scala and Java code analyzers to generate SCG metadata.
- OrientDB embedded graph database to support all queries.
- Graph Buddy Server to support operations needed for visualizations and visual code browsing.
- Graph Buddy Visualization library to provide extracted standard visualization capabilities for both VS Code and IntelliJ plugins.
- VS Code and IntelliJ plugins to natively extend IDE's capabilities with Graph Buddy functionality.

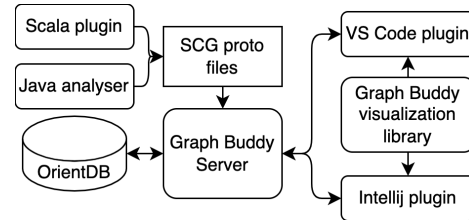


Fig. 1. Graph Buddy architecture – SCG protobuf files are the source information model for visualization.

B. Graph Buddy user interface

The Graph Buddy integrates closely with the IDE source code editor. Each definition marked by the cursor in the source code file is drawn on the interactive Graph Buddy board (Fig. 2). Furthermore, by clicking on a graph node or edge, the appropriate place in the source code file will be highlighted. From the context menu, it is possible to display the call hierarchy (with a chosen depth of recursive neighbors), find a path between nodes, or find only specific dependencies. Users can also filter out some types of nodes or edges, choose a predefined layout (classical, horizontal or vertical) or adjust it manually, delete or search through the displayed nodes.

⁴<https://marketplace.visualstudio.com/items?itemName=virtuslab.graph-buddy>

⁵<https://plugins.jetbrains.com/plugin/13467-graph-buddy>

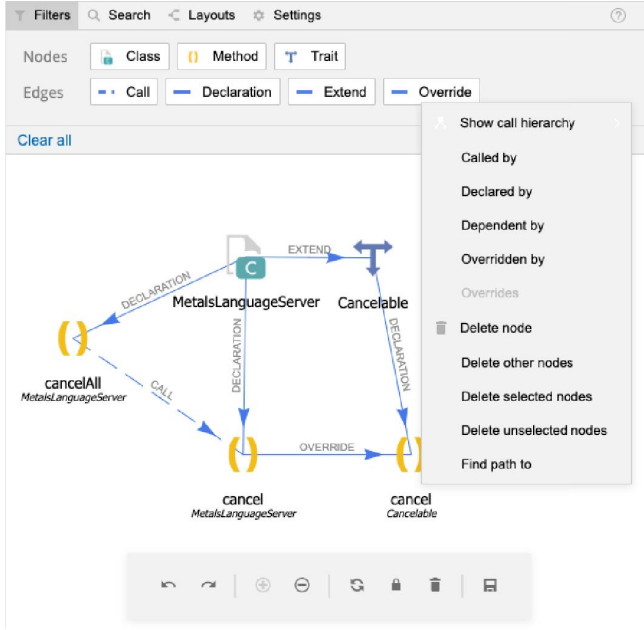


Fig. 2. Graph Buddy user interface with partial graph visualization of `MetalsLanguageServer` class – its two declared methods, `cancelAll` directly calling `cancel`, and method `cancel` overriding method defined in `Cancellable` trait extended by `MetalsLanguageServer`. The user can interactively inspect the related source code by selecting the node or edge, analyze additional dependencies through a context menu, or filter nodes or edges by their type.

Visualizations can be stored in a separate file for later use or sharing with other users.

Changes to the source code require regenerating related SCG meta files by running regeneration process (Java), or by recompiling the project (possible incremental compilation) (Scala), and refreshing the server.

IV. GRAPH BUDDY EVALUATION

A. Graph Buddy performance

To validate the performance of Graph Buddy, we configured and compiled one of the largest and most popular Scala open source projects, the Akka library⁶. Akka contains 8,120 files and 243,193 lines of code. The *Semantic Code Graph* generated for this project has 260,607 nodes and 796,939 edges. It took 60 seconds for the plugin to load the *Semantic Code Graph* data on the server (macOS, 2.3 GHz Intel Core i7 with 32 GB RAM) with a maximum memory usage of 2GB. The plugin was perfectly responsive despite the significant number of nodes in the graph. This proves that the method is scalable to large code bases. Note that Graph Buddy, while fully operational, is still in the early development phase and advanced performance optimizations have yet to be applied.

B. Graph Buddy user feedback loop

Graph Buddy users can raise the feedback through the score and comments on the plugin's marketplace. At the time of

writing, the VS Code plugin was downloaded more than 2.8k times and has the score 4/5, IntelliJ plugin was downloaded about 5.5k times with the score 4.1/5. The user can also post issues and seek help through the official Github project repository.⁷ If the user agrees, the plugin will also collect anonymous events generated by Graph Buddy. Analyzing these events gives information on active users, the most likely used features, or potential usability problems. Establishing a short user feedback loop, although often overlooked by researchers, is a great opportunity to evaluate the usefulness of the tool and introduce the necessary changes.

C. Graph Buddy usability user survey

In addition to the existing user feedback loop, to validate the usefulness of our solution, we prepared the following experiment using the Graph Buddy plugin. We asked ten programmers (volunteers within VirtusLab with professional experience from 2 to 10 years) to perform three programming-related tasks on the *metals* repository⁸ – which was chosen as a typical medium-sized open source product repository, a close example to what the user might encounter in day to day work – using the Graph Buddy plugin inside their favorite IDE:

- 1) **T1 Finding a call hierarchy.** An *Endpoint* in the *metals* project is a method annotated with `@JsonRequest`. Find all the endpoints that indirectly call the `ReferenceProvider#sortByLocationPosition` method. **Rationale.** Analyzing the call hierarchy of a given method allows assessing which parts of the system will be affected after changing the method implementation.
- 2) **T2 Finding a call path.** Find the shortest call path between `MetalsLanguageServer#rename` and `SymbolAlternatives#isVarSetter`. **Rationale.** Analyzing call path can quickly reveal whether a given place in the code can be affected by a given internal method.
- 3) **T3 Learning about the project from Graph Buddy visualization.** Load a visualization saved in *metals-graph.json* file (Fig. 3) into Graph Buddy. Learn about *metals code actions* and describe the knowledge you can gain from this visualization. **Rationale.** Programmers often need to explain the dependencies of the source code to the team member unfamiliar with the project.

Tasks **T1** and **T2** represent common and important *reachability questions* problems [4]. For task **T1** the preferred solution (Fig. 4) using the Graph Buddy tool required first localizing the definition of `ReferenceProvider#sortByLocationPosition` in the source code and then displaying the call graph hierarchy with the Graph Buddy "Show call graph hierarchy" context menu option. As a result, the user gets an interactive call hierarchy graph with endpoints marked with blue dots (methods which are not called directly in the code) – *definition*, *references*, and *rename*. The user can also follow a particular call path easily – selecting the node highlights its definition in the source code, and selecting the edge highlights the method

⁷<https://github.com/VirtusLab/graphbuddy>

⁸<https://scalameta.org/metals>

⁶<https://github.com/akka/akka>

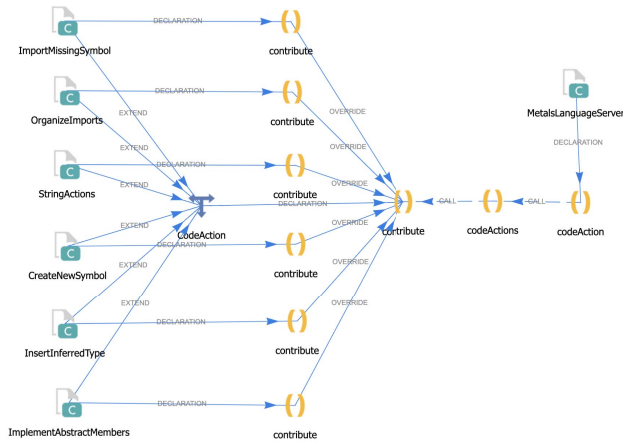


Fig. 3. The dependencies of available code actions in the *metals* library presented in Graph Buddy

invocation. By contrast, alternatively the user can use the *show call hierarchy* feature built in IDEs (both in IntelliJ and VS Code). The results are then presented as a flat expandable tree of all call paths (Fig. 4). However, this representation has significant drawbacks:

- it is hard to localize unique endpoints as endpoints are displayed multiple times, separately for each branched call path,
- it is hard to follow particular call graph as each line represents only call edges, and finding node definitions requires manual code browsing,
- it is hard to spot the additional properties of the code graph otherwise clearly visible in Graph Buddy representation (e.g., the *rename* method having three different possible call paths to *sortByLocationPosition*).

Task **T2** focuses on finding a possible path between two nodes. With Graph Buddy, it was easily possible to find both nodes and use the “Find path to” feature from the context menu. IDEs, on the other hand, do not have a similar functionality (as of writing this article); the best option⁹ is to use again the *Call Hierarchy* feature and manually follow each path, starting from the *SymbolAlternatives#isVarSetter* method, searching for the *MetalsLanguageServer#rename* endpoint (Fig. 5). The main drawback of this solution is that the process is prone to errors and slow. In the worst-case scenario, when the path does not exist, it requires manually going through each possible call graph from the starting method, which in bigger projects might be impossible to complete in a reasonable time.

In task **T3**, the user had to browse through prepared visualization of the code. It turned out that the user was able to quickly list all the implemented actions, spot the common *CodeAction* trait with *contribute* method, read the

⁹<https://stackoverflow.com/questions/34841440/intellij-find-code-path-between-two-methods>

Q	Yes	No	Success rate
Q1	8	2	80%
Q2	10	0	100%
Q3	9	1	90%

TABLE I

GRAPH BUDDY EVALUATION SURVEY’S RESULTS

actual implementation and localize the main method usage. It proves that proper interactive visualization can describe parts of the system efficiently, without the need for additional verbal explanation.

After completing these three tasks, we prepared a short survey:

- **Q1** Did Graph Buddy help you find and understand the call hierarchy?
- **Q2** Did Graph Buddy help you find and understand the call path between two methods?
- **Q3** Were you able to complete the tasks?

The survey results, presented in Table I, strongly indicate that the users found Graph Buddy helpful in solving tasks **T1** and **T2**. Analysis of the graph in task **T3** allowed participants to easily comprehend the structure of the code of *metals* actions. Furthermore, all participants agreed that tasks **T1** and **T2** would be difficult to solve without the Graph Buddy plugin.

V. RELATED WORK

Much of existing approaches are limited to visualizing call graphs [5]–[7]. As such, tools based entirely on call graphs are missing other important code dependencies and entities, especially those language-specific.

Shu et al. [8] present JavaPDG, a *Platform for Program Dependency Analysis*, which focuses on representing and visualizing various dependencies of code statements, such as *system dependence graph*, *procedure dependence graph*, *control flow graph* and *call graph*, extracted from Java bytecode. The created tool can help discover program dependencies, but on a very granular low statement level.

JavaRelationshipGraphs [9] not only focuses on call graphs, but also takes into account other call relationships (such as inheritance, overrides, ownership). The extracted graph is then visualized with the help of the *Neo4j* graph database. Compared to JavaRelationshipGraphs, SCG can represent all the semantic dependencies of the code and, at the same time, bind them to the exact places in the source code.

Although all of the listed approaches have been proven to be useful, none are widely popularized in daily programmers’ workflows. Moreover, most of the tools presented are not publicly available and lack a broader evaluation from a practical angle of the end users.

VI. CONCLUSIONS AND FUTURE WORK

We demonstrated Graph Buddy – an interactive code browsing tool integrated with popular IDEs built on top of the *Semantic Code Graph* information model for representing detailed source code semantic dependencies. The plugins are publicly available with open channels for direct user

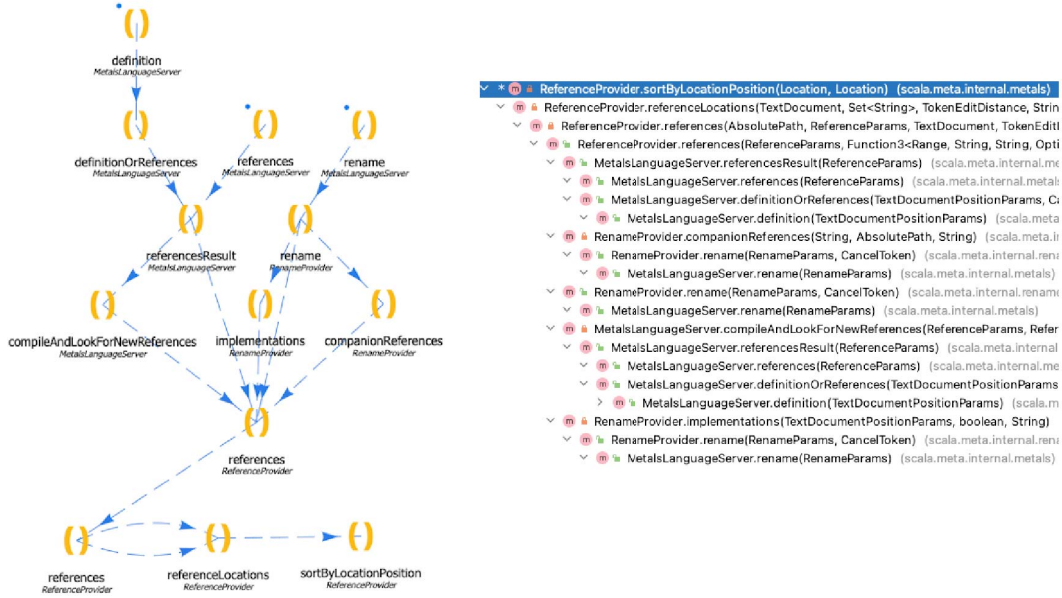


Fig. 4. Graph Buddy (left) vs IntelliJ's call graph solution to task T1.



Fig. 5. Graph Buddy (left) vs IntelliJ find a path solution to task T2. The text is highlighted artificially to mark the possible call path between the *rename* and *isVarSetter* methods.

feedback. Our initial usefulness evaluation proved that with Graph Buddy it is possible to browse and discover vital code dependencies in a more efficient way than with standard IDE features, even on large projects.

In the future, we want to focus on user feedback and analyze available metrics to evaluate possible directions to enable massive plugin adoption. Also, we plan to use *Semantic Code Graph* to explore various new promising ideas, such as advanced semantic-aware code search, code architecture quality monitoring, visualization of the project evolution, or providing advanced semantic refactoring tips.

ACKNOWLEDGMENT

Krzysztof Borowski was supported in part by AGH University of Science and Technology and the "Doktorat Wdrozeniowy" program of the Polish Ministry of Science and Higher Education, and by the VirtusLab company.

REFERENCES

- [1] A. Bandi, B. J. Williams, and E. B. Allen, "Empirical evidence of code decay: A systematic mapping study," in *2013 20th Working Conference on Reverse Engineering (WCORE)*, 2013, pp. 341–350.
- [2] L. Bedu, O. Tinh, and F. Petrillo, "A tertiary systematic literature review on software visualization," in *2019 Working Conference on Software Visualization (VISOFT)*, 2019, pp. 33–44.
- [3] V. Arora, R. K. Bhatia, and M. P. Singh, "Evaluation of flow graph and dependence graphs for program representation," *International Journal of Computer Applications*, vol. 56, pp. 18–23, 2012.
- [4] T. D. LaToza and B. A. Myers, "Developers ask reachability questions," in *2010 ACM/IEEE 32nd International Conference on Software Engineering*, vol. 1, May 2010, pp. 185–194.
- [5] —, "Visualizing call graphs," in *2011 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 2011, pp. 117–124.
- [6] M. D. Shah and S. Z. Guyer, "An interactive microarray call-graph visualization," in *2016 IEEE Working Conference on Software Visualization (VISOFT)*, Oct 2016, pp. 86–90.
- [7] R. Alanazi, G. Gharibi, and Y. Lee, "Facilitating program comprehension with call graph multilevel hierarchical abstractions," *Journal of Systems and Software*, vol. 176, p. 110945, 2021.
- [8] G. Shu, B. Sun, T. A. D. Henderson, and A. Podgurski, "JavaPDG: A New Platform for Program Dependence Analysis," in *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*, 2013, pp. 408–415.
- [9] R. Arora and S. Goel, "JavaRelationshipGraphs (JRG): Transforming java projects into graphs using neo4j graph databases," in *Proceedings of the 2nd International Conference on Software Engineering and Information Management*, ser. ICSIM 2019. ACM, 2019, p. 80–84.